

OPENNEBULA NETWORKING

The purpose of this labsheet is to familiarize yourselves with the Networking infrastructure provided by OpenNebula. We will concentrate on this labsheet in Open vSwitch and we will configure the network interfaces necessary to give full network access to all the Virtual Machines of the Cloud.

OVS - INSTALLATION

OpenvSwitch is a production quality, multilayer virtual switch licensed under the open source Apache 2.0 license. It is designed to enable massive network automation through programmatic extension, while still supporting standard management interfaces and protocols (e.g. NetFlow, sFlow, IPFIX, RSPAN, CLI, LACP, 802.1ag). In addition, it is designed to support distribution across multiple physical servers. We will need to install and configure it to fit our needs.

In order to install this, please download [openvswitch-2.15.0.tar.gz](https://openvswitch.org/download-releases). As you can see this is a tar.gz file.

Go to the *Downloads* folder where you downloaded the tar and extract it. Change into the extracted directory. We will be creating an rpm and to do this the first step is to install the build prerequisites. Use the command:

```
yum install gcc make python-devel kernel-devel graphviz python-sphinx
yum install kernel-debug-devel autoconf automake rpm-build openssl-devel
yum install redhat-rpm-config selinux-policy-devel libtool unbound-devel
yum install python36 python3-sphinx libunwind-devel
```

Once this completes successfully then you will need to run the following:

```
./boot.sh
./configure
make dist
```

Now you have a distribution tarball, named something like openvswitch-x.y.z.tar.gz. Copy this file into the RPM sources directory. You will need to create this directory first:

```
mkdir -p ~/rpmbuild/{SOURCES, BUILD, RPMS, SPECS, SRPMS}
then
cp openvswitch-2.15.0.tar.gz $HOME/rpmbuild/SOURCES
```

While making sure you are still located in the extracted directory openvswitch-2.15.0 (use pwd) you need to build the OpenVSwitch userspace. To do this first you need to run:

```
rpmbuild -bb rhel/openvswitch.spec
```

This will take a while and produces three RPMs: openvswitch, openvswitch-devel and openvswitch-debuginfo in the ~/rpmbuild/RPMS/x86_64. Naturally you are interested in the 1st one.

The desired RPM is now created and you can install it. You could just run `rpm -i` however we find it's better to run:

```
yum localinstall ~/rpmbuild/RPMS/x86_64/openvswitch-2.15.0-1.x86_64.rpm
```

This way any needed system dependencies will automatically resolve themselves. If this command runs its course successfully you will have installed OVS! You can now enable it to run at system startup and start it up so that you can build your virtual networks

```
systemctl enable openvswitch
systemctl start openvswitch
```

To verify it is installed and running correctly run the command: `ovs-vsctl show`

This should return with the ovs UUID and the version of OVS installed. The ovs commands can be seen by running: **man ovs-ctl**. Have a look around and familiarize yourselves with these.

OVS – CONFIGURATION

We are now ready to configure our Cloud network Infrastructure. In order to do this we must build some new modules and virtual interfaces that will now depend on OVS, namely net28, net30, net31 and net32.

We will start by replacing **br0** with **net28**. We will again be bridging this, so we need to create a vmnet28 bridge for the Host to connect to network 28 and hence to the outside world. The commands to do this are:

```
ovs-vsctl add-br vmnet28
ovs-vsctl add-port vmnet28 net28 -- set interface net28 type=internal
ovs-vsctl add-port vmnet28 ens33
ovs-vsctl show
```

Once this is done you can now rename ifcfg-br0 to ifcfg-net28 and then set the contents of it like so:

```
TYPE=Ethernet
BOOTPROTO=none
DEFROUTE=yes
IPV4_FAILURE_FATAL=no
IPV6INIT=yes
IPV6_AUTOCONF=yes
IPV6_DEFROUTE=yes
IPV6_FAILURE_FATAL=no
NAME=net28
DEVICE=net28
ONBOOT=yes
IPADDR=148.197.28.xxx
PREFIX=23
GATEWAY=148.197.28.254
DNS1=148.197.28.251
DOMAIN=netlab.ee.port.ac.uk
IPV6_PEERDNS=yes
IPV6_PEERROUTES=yes
IPV6_PRIVACY=no
NM_CONTROLLED=no
```

In ifcfg-ens33 itself you should only have:

```
DEVICE=ens33
NAME=ens33
BOOTPROTO=none
ONBOOT=yes
TYPE=eth
```

You can now reboot the system (or simply type *service network restart*) and a new ifconfig should reveal a net28 network card with the IP address (instead of br0) and all the info of the default ens33 network card.

In a similar manner you can now create net30:

```
ovs-vsctl add-br vmnet30
ovs-vsctl add-port vmnet30 net30 -- set interface net30 type=internal
ovs-vsctl show
```

and create the **ifcfg-net30** configuration file with the correct IP addresses according to **A2.NET.pdf** file. Repeat the process for **net31** and **net32**. Once all this is done you should restart the network with **service network restart** and you should have an ifconfig on the host that includes all the virtual interfaces and you can use that host by turning it into a router for all the VMs in your Cloud.

NETWORKING IN THE CLOUD

Go to *Infrastructure* then *Virtual Networks* and delete the existing bridge (make sure any running VMs are terminated otherwise you will not be able to delete a network in use), then add a network. In the general tab name it **net28**. In the *conf* tab specify the Bridge as **vmnet28** set the **Network Mode** as **Open vSwitch** and the **VLAN ID** as **No VLAN Network**. In the *addresses* tab specify the **start IP** as your **Prometheus IP +50** (same address you had used for br0) and give it a size of **1**. Finally in the *context* tab set the *network address*, the *subnet mask* the *gateway* and the *DNS* address. Click create.

You will also need to create your net30, 31 and 32 networks. In the general tab name it **net30**. In the *conf* tab specify the Bridge as **vmnet30** set the **Network Mode** as **Open vSwitch** and the **VLAN ID** as **No VLAN Network**. In the *addresses* tab specify the **start IP**. Remember to start from your 2nd useable IP address (because you allocated your first to the host net30 interface) and give it a size of **1**. Finally in the *context* tab set the *network address*, the *subnet mask* the *gateway* and the *DNS* address. Click create. You now have the networks available and you can add one of your VMs or build a new VM using those connections.

DYNAMIC ROUTING

For this task you will need to setup dynamic Routing with OSPF. To do this you need Linux's routing Daemon, Quagga. Type **yum install quagga** and then go to Moodle and fetch the **ROUTING UTILS** folder. Save **zebra.conf** and **ospfd.conf** in **/etc/quagga/** and modify them accordingly for your system i.e. change the network addresses to your own ones. Once this is done you can enable and start the routing daemons:

```
systemctl enable zebra; systemctl enable ospfd; systemctl start zebra; systemctl start ospfd
```

You then need to disable the firewall by typing **iptables -F** and then enable packet forwarding by typing **/sbin/sysctl -w net.ipv4.ip_forward=1**.

You should now be able to see in your routing table (**route -n**) your advertised routes and when you build all your Cloud VMs using these interfaces they should be able to ping google.com. (Please note some students have reported this not working due to VMware NATing limitations on Prometheus. In this case you will still get full marks without the final testing screenshots).